# Data Structures & Algorithms Programming lab
# 6.0CA107C01H

## Manav Rachna International Institute of Research and Studies

## School of Computer Applications

### Department of Computer Applications

| Submitted By | |
|---|---|
| Student Name | Bhupender Raghav |
| Roll No | 26MCAP101N012 |
| Programme | Master of Computer Applications |
| Semester | 1stSemester |
| Section/Group | A |
| Department | Computer Applications |
| Batch | 2026-28 |
| | |
| **Submitted To** | |
| Faculty Name | Dr. Sachin Sharma |

MANAV RACHNA
vidyanatariksha
Manav Rachna International Institute of Research and Studies
Deemed to be University under section 3 of the UGC Act, 1956

**SCHOOL OF COMPUTER APPLICATIONS**

## 1. Write a program to perform various operations in the list: Insertion, Deletion and Display.

### PYTHON PROGRAM

```
a = [10, 20, 30]
print("List:", a)
a.insert(1, 15)
print("After insertion:", a)
a.remove(20)
print("After deletion:", a)
print("List:", a)
```

### OUTPUT

```
List: [10, 20, 30]
After insertion: [10, 15, 20, 30]
After deletion: [10, 15, 30]
List: [10, 15, 30]
```

## 2. Write a program to sort a list using Quick Sort and Merge Sort. Compare their time complexity.

### PYTHON PROGRAM

```
def quick(a):
    if len(a) <= 1:
        return a
    p = a[0]
    return quick([x for x in a[1:] if x <= p]) + [p] + quick([x for x in a[1:] if x > p])

def merge(a):
    if len(a) <= 1:
        return a
    m = len(a) // 2
    l, r = merge(a[:m]), merge(a[m:])
    result = []
    while l and r:
        if l[0] < r[0]:
            result.append(l.pop(0))
        else:
            result.append(r.pop(0))
    return result + l + r

a = [34, 12, 5, 66, 21]
print("Quick Sort:", quick(a))
print("Merge Sort:", merge(a))
print("Quick Sort: Average O(n log n), Worst O(n^2)")
print("Merge Sort: O(n log n)")
```

### OUTPUT

```
Quick Sort: [5, 12, 21, 34, 66]
Merge Sort: [5, 12, 21, 34, 66]
Quick Sort: Average O(n log n), Worst O(n^2)
Merge Sort: O(n log n)
```

## 3. Write programs for Stack, Queue and Circular Queue using arrays, and implement browser history using stacks.

**PYTHON PROGRAM**

```
stack = []
stack.append("A")
stack.append("B")
print("Stack:", stack)
print("Pop:", stack.pop())

queue = []
queue.append(10)
queue.append(20)
print("Queue:", queue)
print("Dequeue:", queue.pop(0))

q = [None, None, None]
front = rear = count = 0
def enqueue(x):
    global rear, count
    if count < 3:
        q[rear] = x
        rear = (rear + 1) % 3
        count += 1
def dequeue():
    global front, count
    if count:
        x = q[front]
        front = (front + 1) % 3
        count -= 1
        return x

enqueue(1)
enqueue(2)
print("Circular Queue:", dequeue())

history = []
history.append("google.com")
history.append("openai.com")
print("Back:", history.pop())
```

**OUTPUT**

```
Stack: ['A', 'B']
Pop: B
Queue: [10, 20]
Dequeue: 10
Circular Queue: 1
Back: openai.com
```

## 4. Write a program to evaluate a Postfix expression.

**PYTHON PROGRAM**

```
s = []
expr = "5 3 2 * +"
for x in expr.split():
    if x.isdigit():
        s.append(int(x))
    else:
        b = s.pop()
        a = s.pop()
        if x == "+":
            s.append(a + b)
        elif x == "-":
            s.append(a - b)
        elif x == "*":
            s.append(a * b)
        elif x == "/":
            s.append(a // b)
print("Postfix:", expr)
print("Output:", s.pop())
```

**OUTPUT**

```
Postfix: 5 3 2 * +
Output: 11
```

## 5. Write a program to convert infix notation into postfix notation.

### PYTHON PROGRAM

```
def convert(s):
    stack = []
    out = ""
    p = {"+": 1, "-": 1, "*": 2, "/": 2}
    for c in s:
        if c.isalnum():
            out += c
        elif c == "(":
            stack.append(c)
        elif c == ")":
            while stack and stack[-1] != "(":
                out += stack.pop()
            stack.pop()
        else:
            while stack and stack[-1] != "(" and p.get(stack[-1], 0) >= p[c]:
                out += stack.pop()
            stack.append(c)
    while stack:
        out += stack.pop()
    return out

print("Infix: (A+B)*C")
print("Output:", convert("(A+B)*C"))
```

### OUTPUT

```
Infix: (A+B)*C
Output: AB+C*
```

## 6. Write a program to perform insertion, deletion and display operations on a linked list.

### PYTHON PROGRAM

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

head = None
def insert(x):
    global head
    n = Node(x)
    n.next = head
    head = n

def delete(x):
    global head
    p = None
    c = head
    while c and c.data != x:
        p, c = c, c.next
    if c:
        if p:
            p.next = c.next
        else:
            head = c.next

def display():
    c = head
    while c:
        print(c.data, end=" -> ")
        c = c.next
    print("None")

insert(30)
insert(20)
insert(10)
display()
delete(20)
display()
```

**OUTPUT**

```
10 -> 20 -> 30 -> None
10 -> 30 -> None
```

## 7. Write a program to perform insertion, deletion and display operations on a circular linked list.

**PYTHON PROGRAM**

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

last = None
def insert(x):
    global last
    n = Node(x)
    if last is None:
        n.next = n
    else:
        n.next = last.next
        last.next = n
    last = n

def delete(x):
    global last
    if last is None:
        return
    p, c = last, last.next
    while c.data != x and c != last:
        p, c = c, c.next
    if c.data == x:
        if c == last and c.next == last:
            last = None
        else:
            p.next = c.next
            if c == last:
                last = p

def display():
    if last is None:
        print("Empty")
        return
    c = last.next
    while True:
        print(c.data, end=" -> ")
        c = c.next
        if c == last.next:
            break
    print("Start")

insert(10)
insert(20)
insert(30)
display()
delete(20)
display()
```

**OUTPUT**

```
10 -> 20 -> 30 -> Start
10 -> 30 -> Start
```

## 8. Write programs for Stack, Queue and Circular Queue using a Linked List.

**PYTHON PROGRAM**

```
class Node:
    def __init__(self, x):
        self.data = x
        self.next = None

top = None
def push(x):
    global top
    n = Node(x)
    n.next = top
    top = n
def pop():
    global top
    x = top.data
    top = top.next
    return x
push(10)
push(20)
print("Stack Pop:", pop())

front = rear = None
def enqueue(x):
    global front, rear
    n = Node(x)
    if rear is None:
        front = rear = n
    else:
        rear.next = n
        rear = n
def dequeue():
    global front, rear
    x = front.data
    front = front.next
    if front is None:
        rear = None
    return x
enqueue(1)
enqueue(2)
print("Queue Dequeue:", dequeue())

rear = None
def add(x):
    global rear
    n = Node(x)
    if rear is None:
        n.next = n
    else:
        n.next = rear.next
        rear.next = n
    rear = n
def remove():
    global rear
    f = rear.next
    x = f.data
    if f == rear:
        rear = None
    else:
        rear.next = f.next
    return x
add(7)
add(8)
print("Circular Queue:", remove())
```

**OUTPUT**

```
Stack Pop: 20
Queue Dequeue: 1
Circular Queue: 7
```

## 9. Write a program to concatenate two linked lists and represent/add polynomials using linked lists.

**PYTHON PROGRAM**

```
a = [1, 2, 3]
b = [4, 5]
print("Concatenation:", a + b)

p = {2: 3, 1: 2, 0: 1}
q = {2: 4, 0: 5}
r = p.copy()
for e, c in q.items():
    r[e] = r.get(e, 0) + c
print("Polynomial Sum:", r[2], "x^2 +", r[1], "x +", r[0])
```

**OUTPUT**

```
Concatenation: [1, 2, 3, 4, 5]
Polynomial Sum: 7 x^2 + 2 x + 6
```

## 10. Write a program to perform insertion, deletion and display operations on a doubly linked list.

**PYTHON PROGRAM**

```
class Node:
    def __init__(self, x):
        self.data = x
        self.prev = None
        self.next = None

head = None
def insert(x):
    global head
    n = Node(x)
    n.next = head
    if head:
        head.prev = n
    head = n

def delete(x):
    global head
    c = head
    while c and c.data != x:
        c = c.next
    if c is None:
        return
    if c.prev:
        c.prev.next = c.next
    else:
        head = c.next
    if c.next:
        c.next.prev = c.prev

def display():
    c = head
    while c:
        print(c.data, end=" <-> ")
        c = c.next
    print("None")

insert(30)
insert(20)
insert(10)
display()
delete(20)
display()
```

**OUTPUT**

```
10 <-> 20 <-> 30 <-> None
10 <-> 30 <-> None
```